

# Room XI Connect – Final Developer Package

Includes: Developer Handoff (Rev.3), Wireframes (Rev.3), and Comprehensive Gemini App Brief.

---

Room XI Connect helps youth reflect on their emotions, discover local programs, and engage with safe community. It blends daily emotional check-ins, anonymous attendance tracking, and an AI companion (Ximi) who encourages reflection.

---

# ■ Room XI Connect - Developer Handoff (Rev. 3)

\_Prepared for The Room 11 Foundation\_  
\_Developer-Oriented Edition (with inline code examples)\_

This document serves as the final technical specification and handoff for the Room XI Connect application, synthesizing all project requirements and design decisions.

---

## ■ Contents

1. [Overview & Goals](#overview--goals)
2. [Authentication & Session System](#authentication--session-system)
3. [Timezone-Safe Streak Logic (DST fixed)](#timezone-safe-streak-logic-dst-fixed)
4. [XID Privacy Attendance System](#xid-privacy-attendance-system)
5. [Feature Flag System](#feature-flag-system)
6. [Ximi AI Integration (Gemini)](#ximi-ai-integration-gemini)
7. [Admin Dashboard](#admin-dashboard)
8. [Database Schema + Migrations](#database-schema--migrations)
9. [Testing & Launch Checklist](#testing--launch-checklist)
10. [Migration Path to Firebase](#migration-path-to-firebase)

---

## 1. Overview & Goals

Room XI Connect is a **trauma-informed, mental wellness application** designed for youth, focusing on daily mood tracking and emotional support.

- \* **Security & Privacy:** Implement healthcare-grade security (HIPAA/GDPR-compliant) with robust authentication and data encryption.
- \* **Reliability:** Ensure core functionality (check-ins, journaling) is available offline and synchronized reliably when online.
- \* **Scalability:** Utilize PostgreSQL (via **Supabase**) for a scalable, secure backend with Row Level Security (RLS).
- \* **User Experience (UX):** Deliver a calm, non-overwhelming, and engaging youth-first interface.

---

## 2. Authentication & Session System

**Goal:** Secure youth login using server-side sessions (no JWT), stored in PostgreSQL.

### Endpoints

| Method | Path               | Description                              |
|--------|--------------------|------------------------------------------|
| POST   | /api/auth/register | Register user (email, password, profile) |
| POST   | /api/auth/login    | Authenticate user and create session     |
| POST   | /api/auth/logout   | Destroy session                          |
| GET    | /api/profile       | Return current user profile              |

### Backend Setup (Express Session)

The application uses `express-session` with `connect-pg-simple` to store session data directly in a PostgreSQL database.

```
``typescript
// server/app.ts
import session from "express-session";
import pgSession from "connect-pg-simple";

// ... other imports

app.use(
  session({
    store: new (pgSession(session))({
      connectionString: process.env.DATABASE_URL,
      tableName: "sessions",
    }),
  })
);
```

```

    }},
    secret: process.env.SESSION_SECRET, // MUST be a strong, random key
    resave: false,
    saveUninitialized: false,
    cookie: {
      httpOnly: true, // Prevents XSS via JavaScript access
      sameSite: "lax", // CSRF protection
      maxAge: 1000 * 60 * 60 * 24 * 7, // 7 days
    },
  })
};

```

### Authentication Middleware

```

``typescript
// server/middleware/auth.ts
import { Request, Response, NextFunction } from 'express';

export function requireAuth(req: Request, res: Response, next: NextFunction) {
  // Check for the user object set by the session middleware
  if (req.session?.user?.id) {
    return next();
  }
  return res.status(401).json({ error: "Not authenticated" });
}

// Usage Example:
// router.get('/profile', requireAuth, (req, res) => { ... });

```

---

## 3. Timezone-Safe Streak Logic (DST fixed)

**\*\*Goal:\*\*** Compute daily check-in streaks safely across Daylight Saving Time (DST) changes and different timezones

### Frontend (Timezone Detection)

The user's timezone is auto-detected on first login and stored in the `profiles` table to ensure server-side consistency

```

``typescript
// src/lib/timezone.ts

// Auto-detect timezone on login:
const tz = Intl.DateTimeFormat().resolvedOptions().timeZone;
// API call to store:
// await api.updateProfile({ timezone: tz });

```

### Server Logic (Luxon Implementation)

The streak logic uses **\*\*Luxon\*\*** to handle date-time comparisons based on calendar days within the user's local timezone

```

``typescript
// server/services/streak.ts
import { DateTime } from "luxon";

/**
 * Calculates the new streak count based on the user's timezone.
 * @param {object} profile - Contains lastCheckinDate (ISO string) and streakCount.
 * @param {string} checkinTs - The current check-in timestamp (ISO string).
 * @returns {number} The new streak count.
 */
function calcStreak(profile, checkinTs) {
  // Default to a known timezone if not set
  const tz = profile.timezone || "America/Edmonton";

  // Create Luxon DateTime objects in the user's timezone
  const last = DateTime.fromISO(profile.lastCheckinDate, { zone: tz });
  const now = DateTime.fromISO(checkinTs, { zone: tz });

  // Calculate difference in calendar days (startOf('day') is the key fix)
  const diff = now.startOf("day").diff(last.startOf("day"), "days").days;

  let newStreak = profile.streakCount || 0;
  if (diff === 0) {

```

```

    // Same day: streak remains unchanged
    newStreak = profile.streakCount;
  } else if (diff === 1) {
    // Consecutive day: increment streak
    newStreak = profile.streakCount + 1;
  } else if (diff > 1) {
    // Missed a day, reset streak
    newStreak = 1;
  }
}

```

```

  return newStreak;
}

```

---

#### ## 4. XID Privacy Attendance System

**\*\*Goal:\*\*** Allow youth to check into programs anonymously using a unique, unlinkable identifier (XID).

##### ### Endpoints

| Method | Path            | Description                         |
|--------|-----------------|-------------------------------------|
| POST   | /api/xid        | Generate XID for authenticated user |
| GET    | /api/xid/me     | Return current XID                  |
| POST   | /api/attendance | Record attendance via XID           |

##### ### Generation Logic

The XID is a 6-character code prefixed with 'X-' (e.g., 'X-AB1234'). The plain code is immediately hashed and stored in the database.

```

``typescript
// server/services/xid.ts
import crypto from "crypto";

```

```

function generateXID() {
  // Restricted character set to avoid confusion (no 0, I, O, 1)
  const chars = "ABCDEFGHJKLMNPQRSTUVWXYZ23456789";
  let code = "X-";
  for (let i=0; i<5; i++) {
    // Opening: "That sounds like a rough one. I know those days."
    // Follow-Up Question: "What's been weighing you down the most today?"
    // Encouragement: "I read a story once that said storms don't last forever. I believe it, because the sun always comes back."
  }
}

```

##### ### Gemini System Prompt Logic (Critical)

The system prompt is the single most important component for enforcing safety and tone.

```

``typescript
// server/services/ximi.ts

```

```

const SYSTEM_PROMPT = `You are Ximi, a kind, trauma-informed AI companion for youth (13-25).
Your primary role is to facilitate the "Living Journal" feature for Room XI Connect.

```

##### RULES:

1. NEVER give medical, psychological, or diagnostic advice.
2. Use plain, real, youth-friendly language (short sentences, everyday words).
3. Sound human, kind, and real-never robotic, formal, or clinical.
4. Your responses must be brief, encouraging reflection, and focused on validating the user's feelings.
5. If the user expresses clear intent to harm themselves or others (e.g., "I'm going to kill myself"), immediately trigger the crisis protocol.

**CRISIS ESCALATION PROTOCOL:** Respond ONLY with the exact phrase: "CRISIS\_DETECTED\_ESCALATE\_NOW" and then immediately end the conversation.

```

`;

```

```

// Server Stub for API Call
app.post("/api/ximi-chat", requireAuth, async (req, res) => {
  const { moodType, note, mode } = req.body;
  if (!req.session.user.profile.ximiConsent) {
    return res.status(403).json({ error: "Consent required" });
  }

  const model = gemini.getGenerativeModel({ model: "gemini-pro" }); // Use gemini-pro or gemini-2.5-flash

  // Build the full prompt including the system prompt, mode, mood, and user note
  const prompt = buildXimiPrompt(moodType, note, mode);
  const result = await model.generateContent(prompt);

  if (result.response.text().includes("CRISIS_DETECTED_ESCALATE_NOW")) {
    // Handle crisis escalation
  }
}

```

```

    // Trigger human intervention flow (e.g., send SMS/email to crisis worker)
    // ...
  }

  res.json({ ximiMessage: result.response.text() });
});

---

## 7. Admin Dashboard

**Goal:** Provide staff with a secure, aggregated view of moods, attendance, flags, and streaks, with strict permissions.

### Endpoints

| Method | Path | Description | Access |
| :--- | :--- | :--- | :--- |
| `GET` | `/api/admin/users` | List users with streaks and roles. | Admin Only |
| `GET` | `/api/admin/moods` | List all check-ins and flagged entries. | Admin Only |
| `GET` | `/api/admin/attendance` | List all program attendance records (XIDs only). | Admin Only |
| `GET` | `/api/admin/mood-summary/:programId` | Aggregated 'stress meter' data for a program. | Admin/Program

### Admin Middleware

```typescript
// server/middleware/admin.ts
export function requireAdmin(req, res, next) {
  // Assumes user role is stored in the session object
  if (req.session?.user?.role === 'admin') {
    return next();
  }
  res.status(403).json({ error: 'Admin access required' });
}

// Usage Example:
// router.get('/admin/users', requireAdmin, (req, res) => { ... });
```

### Front-End Snippet (Admin View)

```typescript
// src/pages/AdminDashboard.tsx
function AdminDashboard() {
  const { data: users } = useFetch("/api/admin/users"); // Custom fetch hook

  return (
    // ... UI setup

    UserStreakMoods

    {users?.map(u => (
      {u.email}
      {u.streak}
      {u.moodCount}
    ))}

  );
}
```

---

## 8. Database Schema + Migrations

The database is built on **PostgreSQL (via Supabase)**, leveraging RLS for data protection.

### Key Tables

| Table | Purpose | Key Columns | Security Note |
| :--- | :--- | :--- | :--- |
| `users` | Login credentials + role. | `id` (UUID), `email`, `password_hash`, `role` | RLS: Only user can view their own info. |
| `profiles` | User info + consent. | `user_id` (FK), `first_name`, `timezone`, `ximi_consent` | RLS: Only user can view their own info. |
| `sessions` | Express-session store. | `sid`, `sess`, `expire` | Server-only access. |

```

```
| `xids` | Anonymous attendance identifiers. | `id` (UUID), `user_id` (FK), `xid_hash`, `checksum` | Server-on-  
| `attendance` | Program attendance records. | `xid_id` (FK), `program_id`, `timestamp` | RLS: Program organiz  
| `mood_checkins` | Daily mood and wellness entries. | `user_id` (FK), `mood_type`, `checkin_date` | RLS: Only
```

### ### SQL Migration Example (Simplified Schema)

```
```sql  
-- V1.0__initial_schema.sql  
  
-- 1. Users Table  
CREATE TABLE users (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  email TEXT UNIQUE NOT NULL,  
  password_hash TEXT NOT NULL,  
  role TEXT DEFAULT 'user'  
);  
  
-- 2. Profiles Table (linked to users, for personal data)  
CREATE TABLE profiles (  
  user_id UUID PRIMARY KEY REFERENCES users(id),  
  first_name TEXT,  
  age INT,  
  city TEXT,  
  timezone TEXT NOT NULL DEFAULT 'America/Edmonton',  
  ximi_consent BOOLEAN DEFAULT FALSE  
);  
  
-- 3. XID Table (anonymous identifier)  
CREATE TABLE xids (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  user_id UUID REFERENCES users(id),  
  xid_hash TEXT NOT NULL,  
  checksum TEXT NOT NULL  
);  
  
-- 4. Attendance Table (links to XID, not User)  
CREATE TABLE attendance (  
  id UUID PRIMARY KEY DEFAULT gen_random_uuid(),  
  xid_id UUID REFERENCES xids(id),  
  program_id UUID, -- Placeholder for program UUID  
  timestamp TIMESTAMPTZ DEFAULT now(),  
  method TEXT  
);
```

---

### ## 9. Testing & Launch Checklist

#### ### Critical Test Cases

| Component     | Test Case                                                                                                     | Expected Result                             |
|---------------|---------------------------------------------------------------------------------------------------------------|---------------------------------------------|
| Auth          | Attempt to access `/api/profile` with an invalid/expired session cookie.                                      | 401 Unauthorized                            |
| Streak Logic  | User checks in at 11:59 PM before DST ends, and again at 1:01 AM after DST starts (a 25-hr gap).              | Streak increments by 1                      |
| XID Privacy   | Admin attempts to query the `attendance` table using RLS to find a user's email.                              | Access denied (no email visible)            |
| Ximi AI       | User inputs a crisis statement. Ximi responds with "CRISIS_DETECTED_ESCALATE_NOW" and the chat UI is updated. | Appropriate crisis response and UI update   |
| Feature Flags | `VITE_ENABLE_XIMI_AI` is set to `false`.                                                                      | The Ximi chat UI component is not rendered. |

#### ### Launch Checklist (Pre-Deployment)

|     |                        |                                                                                                   |
|-----|------------------------|---------------------------------------------------------------------------------------------------|
| [ ] | Security Audit:        | Final penetration test completed on Auth, XID, and Admin endpoints.                               |
| [ ] | RLS Verification:      | All RLS policies on `profiles`, `xids`, and `attendance` verified to be active and correct.       |
| [ ] | Gemini API Key:        | Production Gemini API key secured and configured in environment secrets.                          |
| [ ] | Feature Flags:         | All planned features set to their intended launch state (`VITE_ENABLE_XIMI_AI` should be `true`). |
| [ ] | PHIPA/GDPR Compliance: | Legal sign-off on data handling and consent flows.                                                |

---

### ## 10. Migration Path to Firebase

The current modular design facilitates a future **Phase 2** transition to a Firebase/Google Cloud serverless architecture.

| Component      | Current Implementation | Firebase Equivalent                 |
|----------------|------------------------|-------------------------------------|
| Database       | PostgreSQL (Supabase)  | Firestore (NoSQL Document Database) |
| Authentication | Express Sessions       | Firebase Authentication (JWT)       |

| **\*\*Backend\*\*** | Express Server | **\*\*Cloud Functions for Firebase\*\*** / Cloud Run |  
| **\*\*Ximi AI\*\*** | Express Route + Gemini API | Dedicated Cloud Function + Gemini API |

The backend logic (streak calculation, XID generation) is isolated in services and can be ported to Cloud Functions

---

## ■ Room XI Connect Wireframes

Conceptual Layouts & Flows (Rev. 3)

### ■■ 1. Home / Dashboard (Youth View)

Purpose: First screen after login - summarizes the youth's current streak, mood history, and shortcuts.

Elements

Header: "Hey [FirstName] ■"

Card: "Today's Mood Check-In"

Button → "Check In Now"

Displays current streak count (■ 7-day streak)

Quick Access:

Talk with Ximi (if AI enabled)

My XID (QR badge)

Programs Near Me

Footer Nav:

Home ■ | Programs ■ | Journal ■ | Profile ■

### ■■ 2. Mood Check-In Flow (3-Step Modal)

Component: MoodCheckInFlow.tsx

Trigger: "Check In Now" button on Home

#### Step 1 - Choose Mood

Grid of six weather moods:

| Mood   | Emoji | Label            |
|--------|-------|------------------|
| Cold   | *■    | Numb / Withdrawn |
| Stormy | ■■    | Overwhelmed      |
| Foggy  | ■■    | Confused         |
| Clear  | ■■    | Calm             |
| Breezy | ■     | Upbeat           |
| Aurora | ■     | Amazing          |

Next →

#### Step 2 - Add Details

Tags: "What's influencing your mood?" → physical / emotional / social / school / spiritual

Text area: optional short note ("Anything you want to add?")

Next →

#### Step 3 - Review & Submit

Summary card showing mood + note

Toggle: "Share with staff" (on/off)

Submit button

→ On success:

Toast: "Saved! Want to talk with Ximi?" → opens Ximi Chat

### ■ 3. Ximi Chat Window

Purpose: Reflective AI conversation

Layout

Floating chat panel (bottom right on desktop, full-screen modal on mobile)

Ximi avatar (■)

Message bubbles: warm tone (from Ximi Tone & Dialogue Pack)

Toggle: "Mode - Little Sibling / Peer Guide"

Input box: free-text + send

Safety override: if crisis flagged → show banner:

"If you're in danger or need to talk to someone now, here's help."  
[Kids Help Phone Link] [Hope for Wellness]

#### ■ 4. Profile Page

Purpose: Manage personal data and privacy settings

Sections

Basic Info:

Name, Age, City (editable)

Timezone (auto-detected, editable)

XID Section:

"Get my XID" → shows code (e.g., X-AB1234) + QR code

Tooltip: "Program organizers only see this code, not your name."

Privacy Settings:

Allow Ximi chat ■ toggle

Share moods with staff ■ toggle

Logout button

#### ■■ 5. Programs / Attendance

Purpose: Browse and check into community programs.

List View

Program cards: title, category, next session time, distance

Search bar + filters (e.g., sports, art, leadership)

Button: "Check In with My XID"

Opens QR scanner or manual entry

Check-In Confirmation

"You're marked present for Basketball!"

Shows date/time, using anonymized XID

#### ■ 6. Journal / Mood History

Purpose: Let youth view mood trends & notes.

Layout

7-day streak progress bar

Chart: mood scores (1-6) over time

List of recent check-ins:

Date

Mood icon

Optional note snippet

"Reflect with Ximi" shortcut for any past mood

## ■■■ 7. Admin Dashboard

Access: Only for role: 'admin'

Overview Tabs

Users & Streaks

Table: email | current streak | last check-in

Search by name or city

Mood Check-Ins

Filter by date or mood

Crisis flag alerts (■■)

Attendance

View program check-ins (XID only)

Export CSV

Settings

Feature flag toggles (for testing)

Force data sync

## ■ 8. Auth Flow

Screens

Register

Fields: email, password, first name, age, city

Button → Create Account

Login

Email + Password

Forgot Password (optional later)

Session Handling

HttpOnly cookie (connect.sid) created on login

Auto-redirect to /dashboard

## ■ 9. Error / System States

Offline mode banner: "Working offline. Changes will sync soon."

Empty state:

"No moods yet. Start with your first check-in!"

Server error: friendly retry card

Consent required: "You need to enable Ximi to chat."

## ■■■ 10. Feature Flag Controls (Developer / Admin)

Flag Controls UI Visibility

VITE\_ENABLE\_XIMI\_AI Shows Ximi chat button

VITE\_ENABLE\_XID Enables XID card in Profile

VITE\_ENABLE\_ADMIN\_DASHBOARD Enables Admin route

VITE\_USER\_TIMEZONE\_ENABLED Enables timezone selector

## ■ Navigation Map (Simplified)

Login/Register



↓  
Dashboard (Home)  
    ■■■ Mood Check-In Flow  
    ■     ■■■ Ximi Chat  
    ■■■ Programs  
    ■     ■■■ Attendance (XID)  
    ■■■ Journal  
    ■■■ Profile  
        ■■■ XID  
        ■■■ Settings  
        ■■■ Logout  
(Admins)  
    ■■■ Admin Dashboard

■ Optional UI Notes

Framework: React + Tailwind CSS

Design Language: modern, rounded cards, pastel gradients

Icons: Heroicons or Lucide

Mood colors:

Cold → #A7C7E7

Stormy → #49516D

Foggy → #C9D6DF

Clear → #FDE68A

Breezy → #60A5FA

Aurora → #C084FC

■ Summary

Your current wireframes cover the full youth journey:

Feel → Reflect → Act → Connect.

They're already structured for offline use, privacy protection (XID), and integration with Gemini-powered Ximi.

show me a mock up of the main screens of this app using this design system

■ Room XI Connect - Final Design System

Version: v1.0

Theme: Cosmic Weather Garden

Focus: Trauma-informed, mood-based journaling UX for youth

\*\*Design Layer Only - Backend Remains Unchanged ■\_

■ System Goals

Design Goal      Description

■ Emotional Safety      Calming orbs, soft colors, no overstimulation

■ Cognitive Clarity      Clear hierarchy, high contrast, predictable interactions

■ Youth-first UX      Emojis, mood metaphors, large buttons, no clinical language

■ Offline-First      Fully functional with local IndexedDB; works without styling dependency

■ Privacy-Forward      No PII stored; no visual identifiers tied to identity

■ Color Palette

■ Core Colors (Cosmic Garden Base)

| Token    | Hex     | Use                      |
|----------|---------|--------------------------|
| cream    | #F4EFE6 | Background base          |
| surface  | #FFFDF9 | Cards, modals            |
| deepSage | #2F4A3F | Headings, strong text    |
| sage     | #6E8F7A | Subtext, muted elements  |
| teal     | #2EC489 | Primary buttons, accents |
| gold     | #D8AE3D | Premium features, CTAs   |
| coral    | #E06F5E | Crisis alerts            |
| navy     | #0F1A1C | Overlay/dark backgrounds |
| slate    | #64748b | UI dividers, storm mood  |

■ Weather Mood Orbs

| Mood   | Emoji | Score | Color (Tailwind Token) |
|--------|-------|-------|------------------------|
| Cold   | *■    | 1     | mood.cold = #CBD5E1    |
| Stormy | ■■    | 2     | mood.stormy = #64748b  |
| Foggy  | ■■■   | 3     | mood.foggy = #94a3b8   |
| Clear  | ■■■   | 4     | mood.clear = #fbbf24   |

```

Breezy      ■      5      mood.breezy = #f59e0b
Aurora      ■      6      mood.aurora = #8b5cf6
■ Components
■ MoodOrb

```

```

■■

```

Dynamic bg-mood-{type} class

Includes emoji + pulsing effect

■ CosmicCard

Used for modals, forms, and mood cards

■ CosmicButton

Variant      Classes

Primary      bg-gold text-deepSage shadow-glow-gold

Secondary      bg-teal text-white shadow-glow-teal

Ghost      border border-sage text-sage bg-transparent

Crisis      bg-coral text-white font-bold

Example:

Submit

■■ MoodCheckInModal

How are you feeling today?

Save Check-In

■ Typography

/\* Body \*/

font-family: 'Inter', sans-serif;

/\* Display/Headings \*/

font-family: 'Quicksand', sans-serif;

Tailwind:

```

fontFamily: {
  body: ['Inter', 'sans-serif'],
  display: ['Quicksand', 'sans-serif'],
}

```

■ Animations

breathe (Orb Pulse)

@keyframes breathe {

0%, 100% { transform: scale(1); }

50% { transform: scale(1.05); }

}

.animate-breathe {

animation: breathe 8s ease-in-out infinite;

}

glow (Button Shadow)

@keyframes glow {

0%, 100% { box-shadow: 0 0 0 0 rgba(0, 0, 0, 0); }

50% { box-shadow: 0 0 20px 5px rgba(46, 196, 137, 0.4); }

}

■ Accessibility

All buttons use focus-visible with ring-2 ring-teal ring-offset-2

Motion-safe:

```
@media (prefers-reduced-motion: reduce) {  
  * {  
    animation-duration: 0.01ms !important;  
    transition-duration: 0.01ms !important;  
  }  
}
```

#### ■ Spacing & Radius Tokens

| Radius Class | Value | Use                |
|--------------|-------|--------------------|
| rounded-xl   | 16px  | Buttons, inputs    |
| rounded-2xl  | 24px  | Cards, orbs        |
| rounded-3xl  | 32px  | Modals, sheets     |
| rounded-full | 50%   | Mood orbs, avatars |

#### ■ Usage Summary

Save Check-In

All styling is safe to apply over your existing offline check-in queue and mood streak logic. No backend changes.